How to Talk About This Project in Interviews

The 30-second version (elevator pitch)

“I built an end-to-end data pipeline using about 3 million real NYC taxi trip records. I pulled the raw data with Python, loaded it into Snowflake, transformed it with dbt into a proper staging-and-marts structure, wrote data quality tests, and built a dashboard in Looker Studio. It’s on GitHub with full documentation. I picked this project specifically to get hands-on practice with the tools I’d already gotten certified in, since I wanted something I could actually walk through step by step in an interview, not just talk about abstractly.”

The 2-minute version (when they say “tell me more”)

Structure it as: **what → why → how → what I learned**

1. **What:** “It’s an ETL pipeline — I ingest NYC taxi trip data, clean and model it with dbt, and visualize it. About 3 million rows of real trip data from May 2026.”

2. **Why this dataset:** “I chose NYC taxi data because it’s large enough to be realistic, messy enough to require real cleaning decisions, and it’s a dataset hiring managers actually recognize, so I could talk about the specific data quality issues I ran into rather than a toy dataset that’s already perfectly clean.”

3. **How — walk the architecture:** “Python handles extraction and loading — I used pandas to read the raw parquet files, did some initial cleaning, and batch-loaded it into Snowflake. From there, dbt does all the transformation — I built a staging layer that cleans and types the raw data, then fact and aggregate mart tables on top of that. I also wrote 10 dbt tests to validate data quality, then built three dashboards in Looker Studio on top of the mart tables.”

4. **What I learned / what I’d highlight:** pick 1-2 of the stories below depending on what the interview is probing for.

Specific stories to have ready (pick based on the question)

“Tell me about a bug you had to debug”

The timestamp binding error. “When I first tried to load data into Snowflake, I got an error saying it couldn’t bind timestamp data. It turned out the Snowflake Python connector couldn’t handle pandas Timestamp objects directly in a batch insert — I had to convert them to strings first, then let Snowflake parse them back into proper timestamps on the table side. It was a good reminder that type handling between Python and a SQL warehouse isn’t always automatic, even when the types look compatible on paper.”

“Tell me about a data quality issue you found and how you handled it”

The 9 flagged fare records. This is your strongest story — walk through the full arc: - “I wrote a dbt test to flag fares over $500. It initially caught 39 rows, which seemed like too many, so I looked at the actual data instead of just trusting the threshold.” - “I found two different patterns: some were legitimate long-distance trips — over 100 miles, several hours long, genuinely expensive. Others were fares like $5,525 on a 14-second, 0.4-mile trip — clearly a data error.” - “So I rewrote the test to combine fare *and* distance — flagging high fares only when paired with an implausibly short trip. That brought it down to 9 real anomalies.” - “Then I had a choice: silently filter those 9 rows out, or leave the test failing and document the finding. I chose to leave it failing, because a test that always passes doesn’t actually prove it’s working, and silently dropping rows makes real anomalies invisible to anyone auditing the pipeline later. I documented the reasoning in the README.”

This story demonstrates: not blindly trusting a first-pass threshold, actually investigating data instead of guessing, and a real engineering judgment call about transparency vs. convenience.

“How do you think about data quality/testing?”

“I split my tests into two tiers. Basic completeness checks — not-null, uniqueness — catch data that’s *missing*. But those don’t catch data that’s *present but wrong*. So I added an accepted-values test on payment type, a relationship test to make sure every trip’s location ID actually exists in the zone lookup table, and a custom range check combining fare and distance to catch implausible combinations. That second tier is really where the interesting data quality work happens.”

“Walk me through your dbt project structure”

“I followed a standard staging-then-marts pattern. Staging models do light cleaning — casting types, standardizing column names, filtering obviously bad rows like a corrupted date I found mixed into the data. Marts models sit on top and do the actual business logic — I have a fact table with trip-level calculations like duration and cost-per-mile, and then aggregate tables built for specific dashboard needs, like daily summaries and an hour-by-day breakdown for a peak-demand heatmap.”

“Have you worked with cloud data warehouses?”

“Yes, Snowflake specifically. I set up a database with separate raw, staging, and marts schemas, connected dbt to it via a profile, and did all my transformation work as views in Snowflake. I also had to think about things like batch-loading — I loaded 3 million rows in batches of 10,000 rather than all at once, to keep memory and connection overhead reasonable.”

“Tell me about a time you had to make a judgment call without a clear right answer”

The sum-vs-average heatmap fix. “I built a heatmap showing trip volume by day of week and hour. Initially I just summed everything — but that meant ‘Friday at 6pm’ was actually the total across all five Fridays in May, which inflated the numbers and wasn’t really

representative of a *typical* Friday. I changed it to divide by the number of times each weekday occurred in the month, so it showed a true average. It seems like a small fix, but it changed what the chart was actually claiming to represent, and I wanted to make sure the numbers I was showing were honest, not just impressive-looking."

Questions they might ask that you should be ready for

"Why Snowflake instead of a plain Postgres database?" "Snowflake is what I'd already gotten certified in and it's genuinely industry-standard for cloud data warehousing, especially at scale. It also let me practice the separation-of-concerns pattern of raw/staging/marts schemas, which mirrors how real data teams organize their warehouses."

"Why dbt instead of just writing SQL scripts?" "dbt gives you version control, testing, and documentation for your SQL transformations, plus dependency management — it automatically figures out the right order to build my models based on their `ref()` calls. Writing raw SQL scripts doesn't give you any of that structure."

"What would you do differently if you did this again / what's the biggest limitation?" Good honest answer: "Right now everything runs manually — I'd want to add orchestration, like Airflow or dbt Cloud's scheduler, so it runs on a schedule instead of me triggering each step by hand. I'd also want to test with a full year of data instead of just one month, since a single month can hide seasonal patterns."

"How would you scale this to more data?" "The pipeline itself would mostly hold up — Snowflake and dbt both handle much larger volumes than 3 million rows without architecture changes. The main thing I'd change is the ingestion step, since right now it's a single Python script doing a full batch load. At real scale I'd want incremental loading — only pulling new data since the last run — rather than reloading everything each time."

Tone/framing tips

- Don't undersell the debugging. Interviewers often care more about *how you diagnosed and fixed problems* than about the fact that the pipeline works.
- Be honest about the 9 failing test rows if asked — don't say "all tests pass," say "9 of 10 pass, and the one that fails is intentional, here's why." This is a strength, not a weakness, if you can explain the reasoning.
- If asked something you genuinely don't know (e.g. deep Snowflake internals, orchestration tools you haven't used), it's fine to say "I haven't worked with that yet, but here's how I'd approach learning it" — don't bluff.
- Have the GitHub link ready to share on request — the README does a lot of the explaining for you.